

Documentación Completa — TFM: Multi-Agente ML Stock Picker

Este documento describe **todo lo que hace el repositorio**, en orden de ejecución, con el nivel de detalle suficiente para que cualquier modificación futura sea fácil de planificar y discutir.

Índice

1. [Visión general del sistema](#)
 2. [Estructura de archivos](#)
 3. [Flujo de ejecución principal](#)
 4. [Configuración global — `environment.py`](#)
 5. [Paso 1 — Descarga de datos](#)
 6. [Paso 2 — Consolidación de fundamentales](#)
 7. [Paso 3 — DataRouter y sector map](#)
 8. [Paso 4 — Builders de features](#)
 9. [Paso 5 — Dataset maestro](#)
 10. [Paso 6 — Diagnóstico sectorial](#)
 11. [Paso 7 — Walk-Forward Backtest histórico](#)
 - [11.1 Generación de folds](#)
 - [11.2 Entrenamiento de agentes \(trainer\)](#)
 - [11.3 Agente Fundamental](#)
 - [11.4 Agente de Valoración](#)
 - [11.5 Agente de Momentum](#)
 - [11.6 Agente de Riesgo \(Bear\)](#)
 - [11.7 Meta-Learner](#)
 - [11.8 OOF anti-leakage](#)
 - [11.9 Backtester y métricas](#)
 - [11.10 Explicabilidad SHAP](#)
 12. [Paso 8 — Fold live out-of-sample](#)
 13. [Resultados y ficheros de salida](#)
 14. [Mecanismos anti-leakage](#)
 15. [Cómo modificar el sistema](#)
-

1. Visión general del sistema

El sistema es un **selector de acciones basado en múltiples agentes de Machine Learning** que combina cuatro perspectivas de análisis (fundamentales, valoración, momentum, riesgo) y un meta-aprendiz que las ensambla.

El objetivo es predecir si una acción del S&P 500 **superará o no superará** la mediana del mercado en los próximos ~63 días de trading (~1 trimestre). Con esas predicciones se construye una cartera long con los 10 mejores candidatos por trimestre.

Principios de diseño:

- **Modularidad:** cada agente es intercambiable e independiente.
- **Integridad temporal:** ningún dato futuro se filtra al pasado (anti-leakage estricto).
- **Explicabilidad:** cada predicción tiene un desglose SHAP por features.
- **Reproducibilidad:** semilla fija (`RANDOM_SEED = 42`) en todo el pipeline.

2. Estructura de archivos

```
TFM/
├── analyzer.py           # PUNTO DE ENTRADA – orquestador del pipeline
├── run_pipeline.py       # Punto de entrada alternativo con argumentos CLI
├── environment.py        # CONFIGURACIÓN GLOBAL – todos los parámetros
├── DOCUMENTATION.md     # Este archivo
├── requirements.txt      # Dependencias Python del proyecto
├──
├── module/              # Lógica de negocio
│   ├── __init__.py
│   ├── fetcher.py       # Descarga de datos via yfinance + CSV datasets
│   ├── data_router.py   # Cargador de datos con alineación temporal
│   ├── feature_engineering.py # 4 builders de features + SectorNormalizer
│   ├── backtester.py    # Walk-forward backtesting + métricas financieras
│   ├── explainer.py     # Explicaciones SHAP por agente y ticker
│   ├── visualizer.py    # Gráficos (8 tipos de plots)
│   ├──
│   ├── agents/         # Los 5 modelos del ensemble
│   │   ├── __init__.py
│   │   ├── base_agent.py # Clase base abstracta (contrato común)
│   │   ├── fundamental_agent.py # XGBoost sobre ratios financieros
│   │   ├── valuation_agent.py # GBM sobre múltiplos de valoración
│   │   ├── momentum_agent.py # Random Forest sobre indicadores técnicos
│   │   ├── bear_agent.py  # Filtro de riesgo híbrido (reglas + RF)
│   │   └── meta_learner.py # Stacking (LR + GBM) que combina los 4 agentes
│   ├──
│   └── pipeline/       # Orquestación por etapas
│       ├── __init__.py
│       ├── data_ops.py  # ETL: descarga, consolidación, filtrado
│       ├── dataset_builder.py # Construcción del dataset maestro
│       ├── trainer.py   # Entrenamiento de agentes + OOF
│       ├── evaluator.py # Walk-forward loop + SHAP + backtest
│       ├── live_fold.py # Predicción out-of-sample con precios reales
│       └── ablation.py  # Ablation study: contribución marginal de cada
│
└── agente
    ├── data/           # Datos de entrada (generados por fetcher.py)
    │   ├── companies.csv # Mapa ticker → sector/industria/market_cap
    │   ├── ticker_history.json # Registro de cuándo se actualizó cada ticker
    │   ├── prices/      # OHLCV diario por ticker (un CSV por ticker)
    │   ├── consolidated/ # Fundamentales trimestrales procesados
    │   └── fundamentals/ # Fundamentales crudos de yfinance
```

```

├── analyst/           # Estimaciones de analistas
├── insider/           # Transacciones de insiders
├── macro/             # Indicadores macro (VIX, tipos, S&P 500)
├── results/           # Salidas del pipeline (generadas en ejecución)
│   ├── master_dataset.csv # Dataset completo con features + labels
│   ├── pipeline.log      # Log completo de la ejecución
│   ├── agents/          # Diagnósticos de los agentes por fold
│   │   ├── fundamental/
│   │   ├── valuation/
│   │   ├── momentum/
│   │   ├── bear/
│   │   └── meta_learner/
│   ├── backtest/        # Métricas del backtest por fold
│   └── plots/           # Gráficos generados

```

3. Flujo de ejecución principal

`analyzer.py` es el único archivo que se ejecuta directamente (`python analyzer.py`). Contiene una función `main()` que ejecuta los pasos en este orden:

También existe `run_pipeline.py` como punto de entrada alternativo con soporte de argumentos CLI:

```

python run_pipeline.py           # ejecución completa
python run_pipeline.py --skip-download # reutiliza datos ya en disco
python run_pipeline.py --skip-backtest  # solo fold live
python run_pipeline.py --tickers AAPL MSFT # subconjunto de tickers

```

1. <code>fetch_tickers()</code>	→ lista de ~300 tickers del S&P 500
2. <code>download_data()</code>	→ descarga precios, fundamentales, macro...
3. <code>DataRouter + sector_map</code>	→ carga datos con lag temporal
4. <code>FeatureBuilders</code>	→ instancia los 4 builders
5. <code>build_master_dataset()</code>	→ construye el dataset (ticker, date) con labels
6. <code>plot_sector_performance()</code>	→ gráfico diagnóstico sectorial
7. <code>run_walkforward_pipeline()</code>	→ backtest histórico completo
8. <code>run_live_fold()</code>	→ predicción out-of-sample real
9. Log resultado final	→ alpha, Sharpe, drawdown

Cada paso delega en el módulo correspondiente de `module/pipeline/`. `analyzer.py` en sí no contiene lógica de negocio, solo coordina.

4. Configuración global — `environment.py`

Todo lo que se quiera cambiar del comportamiento del sistema está en este archivo. Nunca hay que entrar en los módulos para ajustar parámetros.

Flags de ejecución

Variable	Tipo	Default	Efecto
SKIP_BACKTEST	bool	False	Si <code>True</code> , salta el walk-forward y solo hace el fold live
FORCE_DOWNLOAD	bool	False	Si <code>True</code> , re-descarga todo aunque los datos sean recientes
FORCE_TRAINING	bool	False	Reservado para re-entrenar sin forzar descarga (no implementado)
NO_PLOT	bool	True	Redundante: el backend Agg hace que <code>plt.show()</code> sea siempre no-op. Los gráficos siempre se guardan a disco.

Rutas

Todas las rutas son relativas al directorio donde se ejecuta `analyzer.py`:

Variable	Valor	Descripción
DATA_DIR	"data"	Raíz de todos los datos
RESULTS_DIR	"results"	Raíz de todos los resultados
PRICES_DIR	"data/prices"	CSVs de precios diarios
CONSOLIDATED_DIR	"data/consolidated"	Fundamentales procesados
INSIDER_DIR	"data/insider"	Transacciones de insiders
ANALYST_DIR	"data/analyst"	Estimaciones de analistas
MACRO_DIR	"data/macro"	Indicadores macroeconómicos
COMPANIES_FILE	"data/companies.csv"	Mapa ticker → sector
AGENTS_RESULTS_DIR	"results/agents"	Diagnósticos de agentes
BACKTEST_RESULTS_DIR	"results/backtest"	Métricas del backtest
PLOTS_DIR	"results/plots"	Gráficos

Período de análisis

Variable	Valor	Descripción
START_DATE	"2021-01-01"	Inicio del histórico (descarga + training)
END_DATE	"2026-01-01"	Fin del histórico / inicio del fold live
DAYS_UPDATE	90	Días tras los que se considera que los datos están obsoletos

Parámetros anti-leakage

Variable	Valor	Descripción
----------	-------	-------------

Variable	Valor	Descripción
FILING_LAG_DAYS	45	Días de retraso de publicación de informes SEC. Un informe de Q1 (cierra 31-Mar) no está disponible hasta ~15-May
FORWARD_RETURN_DAYS	63	Horizonte del label en días de trading (~1 trimestre bursátil). Se calcula el retorno 63 días hacia adelante
MIN_HISTORY_QUARTERS	4	Mínimo de trimestres históricos que debe tener un ticker para ser incluido
SECTOR_ZSCORE_MIN_PEERS	3	Mínimo de empresas del mismo sector para calcular Z-score sectorial

Parámetros del pipeline ML

Variable	Valor	Descripción
OOF_N_SPLITS	3	Número de folds KFold internos para generar OOF scores del meta-learner

Walk-forward backtesting

Variable	Valor	Descripción
WALKFORWARD_TRAIN_YEARS	1	Mínimo de años de datos de entrenamiento en el primer fold
WALKFORWARD_TEST_QUARTERS	1	Trimestres de test por fold (siempre 1 trimestre = ~63 días)
RISK_FREE_RATE	0.04	Tasa libre de riesgo anualizada para calcular Sharpe y Sortino
TOP_N_STOCKS	10	Número de acciones en la cartera long por fold

Hiperparámetros de los agentes

Todos los modelos tienen sus hiperparámetros definidos aquí. Si se quiere ajustar un modelo, se cambia la constante correspondiente y se re-ejecuta; no hay que tocar el código del agente.

FundamentalAgent (XGBoost):

FUNDAMENTAL_N_ESTIMATORS	= 400	# Árboles
FUNDAMENTAL_MAX_DEPTH	= 5	# Profundidad máxima
FUNDAMENTAL_LEARNING_RATE	= 0.05	# Learning rate
FUNDAMENTAL_SUBSAMPLE	= 0.8	# Fracción de filas por árbol
FUNDAMENTAL_COLSAMPLE	= 0.7	# Fracción de columnas por árbol
FUNDAMENTAL_MIN_CHILD_WEIGHT	= 5	# Regularización (evita sobreajuste en hojas pequeñas)
FUNDAMENTAL_CV_FOLDS	= 5	# Folds para early stopping CV

ValuationAgent (GradientBoosting sklearn):

```
VALUATION_N_ESTIMATORS = 200
VALUATION_MAX_DEPTH = 4
VALUATION_LEARNING_RATE = 0.05
VALUATION_SUBSAMPLE = 0.8 # Fracción de filas por árbol
VALUATION_CV_FOLDS = 5
```

MomentumAgent (Random Forest):

```
MOMENTUM_N_ESTIMATORS = 300
MOMENTUM_MAX_DEPTH = 8
MOMENTUM_MIN_SAMPLES_LEAF = 10 # Evita sobreajuste a patrones de corto plazo
MOMENTUM_CV_FOLDS = 5
```

BearAgent (Random Forest híbrido):

```
BEAR_N_ESTIMATORS = 200
BEAR_MAX_DEPTH = 6
BEAR_CV_FOLDS = 5
BEAR_RULE_WEIGHT = 0.5 # Peso de la capa de reglas en el score final
BEAR_ML_WEIGHT = 0.5 # Peso del modelo ML en el score final
BEAR_HARD_THRESHOLD = 0.90 # Si bear_score > 0.90, el meta-learner fuerza Underperform
```

MetaLearner (Logistic Regression + GBM stacking):

```
META_LR_C = 0.5 # Regularización L2 de la Regresión Logística
META_GBM_N_ESTIMATORS = 150
META_GBM_MAX_DEPTH = 3
META_GBM_LEARNING_RATE = 0.05
META_GBM_SUBSAMPLE = 0.8
META_CV_FOLDS = 5
```

Reproducibilidad

```
RANDOM_SEED = 42 # Semilla global para todos los modelos y splits
```

5. Paso 1 — Descarga de datos

Archivo: `module/pipeline/data_ops.py` → función `download_data()` **Delegado en:** `module/fetcher.py`

El fetcher descarga datos de varias fuentes para cada ticker del universo (~300 acciones del S&P 500). Los datos se guardan en `data/` como CSVs individuales por ticker.

Qué se descarga

Función	Fuente	Destino	Frecuencia
<code>fetch_prices()</code>	yfinance	<code>data/prices/<TICKER>.csv</code>	Diaria (OHLCV + Volume)
<code>fetch_fundamentals()</code>	yfinance	<code>data/fundamentals/<TICKER>.csv</code>	Trimestral
<code>fetch_insider_trades()</code>	datasets/CSV	<code>data/insider/<TICKER>.csv</code>	Por evento
<code>fetch_analyst_estimates()</code>	datasets/CSV	<code>data/analyst/<TICKER>.csv</code>	Por publicación
<code>fetch_macro_data()</code>	yfinance	<code>data/macro/</code>	Diaria (VIX, yield curve, S&P 500)

Lógica de actualización

Cuando `FORCE_DOWNLOAD = False` (por defecto), el sistema **solo descarga si los datos tienen más de `DAYS_UPDATE = 90` días**. Esto se controla mediante `data/ticker_history.json`, que registra la última vez que se actualizó cada ticker y categoría.

Cuando `FORCE_DOWNLOAD = True`, se re-descarga todo sin comprobar fechas.

En el modo normal (False), solo se actualizan automáticamente precios y macro; el resto (fundamentales, insider, analyst) solo se actualiza si han pasado los 90 días.

Lista de tickers

La función `fetch_tickers()` en `fetcher.py` devuelve una lista hardcodeada de ~300 tickers del S&P 500, deduplicada con `dict.fromkeys()`. Si se quiere cambiar el universo de acciones, se modifica esta función.

6. Paso 2 — Consolidación de fundamentales

Archivo: `module/pipeline/data_ops.py` → función `prepare_data()` **Delegado en:** `module/fetcher.py` → clase `FinancialDataConsolidator`

Este paso está comentado en `analyzer.py` por defecto. Solo es necesario ejecutarlo una vez (o cuando se añaden nuevas fuentes de datos). Genera los CSVs en `data/consolidated/` que el resto del pipeline consume.

El `FinancialDataConsolidator` toma los datos crudos de `data/fundamentals/`, `data/analyst/`, `data/insider/` y los fusiona en un único CSV por ticker con columnas estandarizadas y periodicidad trimestral:

- Balance sheet: activos, pasivos, deuda, capital
- P&L: ingresos, beneficio operativo, neto, EPS
- Cash flow: flujo libre (FCF), capex, D&A
- Ratios derivados: ROE, ROA, márgenes

- Estimaciones de analistas: EPS estimado, EPS reportado, sorpresa

El resultado (`data/consolidated/<TICKER>.csv`) es lo que `DataRouter.load_consolidated()` carga posteriormente.

7. Paso 3 — DataRouter y sector map

Archivo: `module/data_router.py` → clase `DataRouter`

El `DataRouter` es la capa de acceso a datos con **alineación temporal anti-leakage**. Se instancia una vez y se pasa a todas las funciones que necesitan datos.

```
router = DataRouter(data_dir=DATA_DIR, filing_lag_days=FILING_LAG_DAYS)
sector_map = router.get_sector_map() # {ticker: sector}
```

Métodos principales

Método	Qué hace
<code>load_companies()</code>	Carga <code>data/companies.csv</code> con ticker, sector, industria, market_cap
<code>get_sector_map()</code>	Devuelve <code>{ticker: sector}</code> para normalización sectorial
<code>load_prices(ticker)</code>	Carga <code>data/prices/<TICKER>.csv</code> → Serie de precios diarios
<code>load_consolidated(ticker)</code>	Carga <code>data/consolidated/<TICKER>.csv</code> con shift de +45 días al índice temporal
<code>load_insider(ticker)</code>	Carga <code>data/insider/<TICKER>.csv</code>
<code>load_analyst(ticker)</code>	Carga <code>data/analyst/<TICKER>.csv</code>
<code>load_macro()</code>	Carga todos los CSVs de <code>data/macro/</code> fusionados (cacheado en memoria)
<code>load_sp500_prices()</code>	Carga el benchmark del S&P 500

Anti-leakage en `load_consolidated`

El método `load_consolidated()` desplaza el índice de fecha de los fundamentales **+45 días hacia adelante**. Esto simula el retraso real de publicación de informes SEC:

- Trimestre Q1 cierra el 31-Mar → el informe es público ~15-May (+45 días)
- Sin este shift, el modelo vería datos financieros del Q1 al principio de Q1, lo cual es imposible en la realidad

Si se quiere cambiar este lag, se modifica `FILING_LAG_DAYS` en `environment.py`.

Filtrado de tickers disponibles

Después de instanciar el router, `get_available_tickers()` filtra la lista a solo los tickers que tienen **ambos archivos en disco**:

- `data/prices/<TICKER>.csv`
- `data/consolidated/<TICKER>.csv`

Los tickers sin datos completos son descartados para evitar errores downstream.

8. Paso 4 — Builders de features

Archivo: `module/feature_engineering.py`

Se instancian 4 builders, uno por dominio de análisis. Cada builder toma los datos crudos del router y genera un DataFrame de features para un ticker y rango de fechas.

```
fundamental_builder = FundamentalFeatureBuilder()
technical_builder   = TechnicalFeatureBuilder()
valuation_builder   = ValuationFeatureBuilder()
insider_builder     = InsiderFeatureBuilder()
```

Los builders no tienen estado persistente entre tickers (son stateless). Se llaman en el `dataset_builder` para cada ticker.

FundamentalFeatureBuilder

Calcula ratios financieros a partir de `data/consolidated/`:

Feature	Descripción
<code>roe, roa, roi, roic</code>	Retorno sobre equity, activos, inversión y capital invertido
<code>net_margin, gross_margin, fcf_margin, ebitda_margin, operating_margin</code>	Márgenes de rentabilidad
<code>current_ratio, quick_ratio</code>	Ratios de liquidez
<code>debt_equity, debt_to_ebitda, interest_coverage</code>	Ratios de apalancamiento
<code>revenue_yoy_growth, net_income_yoy_growth, eps_yoy_growth</code>	Crecimiento interanual
<code>fcf_yoy_growth, operating_income_yoy_growth, total_debt_yoy_growth</code>	Más crecimientos
<code>accruals_ratio</code>	Calidad del beneficio (diferencia beneficio contable vs flujo de caja)
<code>capex_to_revenue</code>	Intensidad de capital

Feature	Descripción
<code>consecutive_losses</code>	Número de trimestres consecutivos con pérdidas

Nota: `ebitda_margin` y `operating_margin` solo se calculan si no existen ya en el DataFrame (para evitar sobrescribir cálculos del consolidador).

TechnicalFeatureBuilder

Calcula indicadores técnicos a partir de `data/prices/`:

Categoría	Features
Osciladores	<code>rsi_14</code> , <code>rsi_28</code>
Tendencia	<code>macd</code> , <code>macd_signal</code> , <code>macd_histogram</code> , <code>sma_20</code> , <code>sma_50</code> , <code>sma_200</code>
Bandas	<code>bollinger_pct</code> (posición relativa en Bollinger Bands)
52 semanas	<code>price_vs_52w_high</code> , <code>price_vs_52w_low</code>
Momentum	<code>momentum_1m</code> , <code>momentum_3m</code> , <code>momentum_6m</code> , <code>momentum_12m</code>
Volatilidad	<code>volatility_20d</code> , <code>volatility_60d</code> , <code>atr_14</code>
Volumen	<code>volume_ratio_20</code> , <code>volume_ratio_50</code>

ValuationFeatureBuilder

Calcula múltiplos de valoración a partir de `data/consolidated/` + `data/prices/` + `data/analyst/`:

Feature	Descripción
<code>pe_ratio</code> , <code>pb_ratio</code> , <code>ps_ratio</code>	P/E, P/B, P/S
<code>ev_ebitda</code> , <code>fcf_yield</code> , <code>earnings_yield</code>	EV/EBITDA, rendimiento FCF, inverso del P/E
<code>pe_vs_5y_median</code> , <code>pb_vs_5y_median</code> , <code>ev_ebitda_vs_5y_median</code>	Comparación con histórico propio de 5 años
<code>eps_surprise_pct</code> , <code>eps_revision</code> , <code>eps_estimate</code> , <code>eps_reported</code>	Datos de consenso de analistas
<code>pe_sector_pct</code> , <code>pb_sector_pct</code> , <code>ev_ebitda_sector_pct</code> , <code>fcf_yield_sector_pct</code>	Percentil sectorial (calculado dinámicamente dentro de cada fold de train)

InsiderFeatureBuilder

Calcula señales de transacciones de directivos a partir de `data/insider/`:

Feature	Descripción
<code>insider_net_shares_90d</code>	Shares compradas menos vendidas en los últimos 90 días

Feature	Descripción
<code>insider_sell_ratio</code>	Ratio de ventas sobre total transacciones (0=solo compras, 1=solo ventas)
<code>insider_transaction_count</code>	Número de transacciones en los últimos 90 días

SectorNormalizer

Clase separada dentro de `feature_engineering.py`. Aplica Z-score sectorial: para cada feature, calcula media y desviación estándar dentro del sector y estandariza. Esto hace comparables empresas del mismo sector y elimina sesgos sectoriales.

Importante: el `SectorNormalizer` se ajusta (`fit=True`) **solo sobre datos de entrenamiento**. En test se aplica la transformación ya ajustada (`fit=False`). Esto evita que información del futuro influya en la normalización del pasado.

9. Paso 5 — Dataset maestro

Archivo: `module/pipeline/dataset_builder.py` → función `build_master_dataset()`

Esta función construye el DataFrame principal del sistema: una tabla multi-índice (`ticker, date`) donde cada fila es una observación trimestral de un ticker con sus features y su label.

```
df = build_master_dataset(
    tickers=tickers_ok,
    router=router,
    fundamental_builder=fundamental_builder,
    technical_builder=technical_builder,
    valuation_builder=valuation_builder,
    insider_builder=insider_builder,
    min_history_quarters=MIN_HISTORY_QUARTERS,    # 4
    forward_return_days=FORWARD_RETURN_DAYS,     # 63
)
```

Proceso para cada ticker

1. Cargar fundamentales consolidados (`router.load_consolidated(ticker)`) — ya con lag +45 días
2. Construir features fundamentales (`fundamental_builder.build(df_consolidated)`)
3. Cargar precios diarios (`router.load_prices(ticker)`)
4. Construir features técnicas (`technical_builder.build(prices)`) — se alinean a las fechas trimestrales
5. Construir features de valoración (`valuation_builder.build(...)`)
6. Cargar datos de insider (`router.load_insider(ticker)`)
7. Construir features de insider (`insider_builder.build(insider_df)`)
8. Cargar macro (`router.load_macro()`) — mismo para todos los tickers
9. Fusionar todos los features en un único DataFrame por ticker
10. Calcular el **label**: retorno de la acción en los próximos `FORWARD_RETURN_DAYS = 63` días de trading

11. Descartar observaciones con menos de `MIN_HISTORY_QUARTERS = 4` trimestres

El label (variable objetivo)

El label es **continuo** (`forward_return`) en el dataset maestro. En el pipeline de entrenamiento se convierte a **binario**: `1` si el retorno supera la mediana del conjunto de train, `0` en caso contrario.

Esta binarización se hace fold a fold, usando siempre la mediana del período de entrenamiento (no del test), para evitar leakage.

Columna `sector`

El router añade la columna `sector` a cada observación según `data/companies.csv`. Esta columna se usa para:

- Normalización sectorial (Z-scores)
- Variables dummies en el meta-learner
- Diagnósticos sectoriales

Salida

El dataset se guarda en `results/master_dataset.csv` y se mantiene en memoria como `df` para los pasos siguientes.

10. Paso 6 — Diagnóstico sectorial

Archivo: `module/visualizer.py` → método `plot_sector_performance()`

Genera un gráfico de rendimiento medio por sector (retorno forward promedio por sector y período). Se guarda en `results/plots/`. Si `NO_PLOT = True`, no se muestra en pantalla.

11. Paso 7 — Walk-Forward Backtest histórico

Archivo principal: `module/pipeline/evaluator.py` → función `run_walkforward_pipeline()`

Este es el paso central del sistema. Simula cómo habría funcionado la estrategia históricamente, entrenando y evaluando en períodos sucesivos para evitar el data snooping.

11.1 Generación de folds

Archivo: `module/backtester.py` → método `generate_folds()`

El backtester genera una lista de "folds" temporales. Cada fold define:

- `train_start`: inicio del período de entrenamiento
- `train_end`: fin del entrenamiento / inicio del test
- `test_end`: fin del período de test
- `_train_years`: duración real del train en años

El esquema es **walk-forward con ventana expandible**:

- Fold 1: train = [2021-01-01, 2022-01-01), test = [2022-01-01, 2022-04-01)
- Fold 2: train = [2021-01-01, 2022-04-01), test = [2022-04-01, 2022-07-01)
- ... y así sucesivamente hasta `END_DATE`

El parámetro `WALKFORWARD_TRAIN_YEARS = 1` determina cuántos años mínimos de train se necesitan para el primer fold.

Verificación de coherencia: el backtester compara `FORWARD_RETURN_DAYS` con el número estimado de días de trading en el período de test. Si difieren más de 15 días, emite un `WARNING` en el log.

11.2 Entrenamiento de agentes (trainer)

Archivo: `module/pipeline/trainer.py` → función `train_fold()`

Para cada fold, `train_fold()` ejecuta el siguiente flujo:

1. Separar `df_train` / `df_test` por fechas
2. Eliminar duplicados de índice (`keep='last'`)
3. Normalización sectorial:
 - fit en train → `SectorNormalizer` aprende medias/stds del train
 - transform en test → aplica la misma normalización
4. Binarizar labels:
 - `median_ret` = mediana de `forward_return` en TRAIN
 - `y_train` = `(forward_return > median_ret).astype(int)`
 - `y_test` = `(forward_return > median_ret).astype(int)` ← misma mediana del train
5. Instanciar los 4 agentes + `MetaLearner`
6. Entrenar agentes base (`FundamentalAgent`, `ValuationAgent`, `MomentumAgent`, `BearAgent`)
7. Generar OOF scores (`_generate_oof_scores`) ← ver sección 11.8
8. Entrenar `MetaLearner` sobre OOF scores
9. Predecir en test con todos los agentes
10. Devolver (`agents_dict`, `df_test_con_scores`, `df_train_with_oof`)

El tercer elemento `df_train_with_oof` contiene las columnas `*_score` de OOF y se pasa al ablation study.

11.3 Agente Fundamental — `FundamentalAgent`

Archivo: `module/agents/fundamental_agent.py`

- **Modelo:** XGBoost clasificador calibrado con `CalibratedClassifierCV` (isotonic regression)
- **Columnas de features:** ~14 fundamentales + sus versiones Z-score sectorial (`_zsector`)
- **Entrenamiento:** CV con early stopping para evitar sobreajuste; guarda AUC train/val en diagnósticos
- **Salida:** Score [0, 1] donde **1 = empresa financieramente sana** → **Outperform**

Features que usa (FEATURE_COLS):

```

roe, roa, roi, roic
net_margin, gross_margin, fcf_margin, ebitda_margin, operating_margin
current_ratio, quick_ratio
debt_equity, debt_to_ebitda, interest_coverage
revenue_yoy_growth, net_income_yoy_growth, eps_yoy_growth
fcf_yoy_growth, operating_income_yoy_growth, total_debt_yoy_growth
accruals_ratio, capex_to_revenue, consecutive_losses

```

Más las versiones `_zsector` de cada una (Z-score dentro del sector), y dummies del sector (`sector_<nombre>`).

11.4 Agente de Valoración — `ValuationAgent`

Archivo: `module/agents/valuation_agent.py`

- **Modelo:** `GradientBoostingClassifier` de scikit-learn (calibrado)
- **Salida:** Score [0, 1] donde **1 = empresa infravalorada → Outperform**

Features que usa (FEATURE_COLS):

```

pe_ratio, pb_ratio, ps_ratio
ev_ebitda, fcf_yield, earnings_yield
pe_vs_5y_median, pb_vs_5y_median, ev_ebitda_vs_5y_median
eps_surprise_pct, eps_revision, eps_estimate, eps_reported
pe_sector_pct, pb_sector_pct, ev_ebitda_sector_pct, fcf_yield_sector_pct

```

Los percentiles sectoriales (`_sector_pct`) se calculan **dentro del método `fit()`** usando solo los datos de train, para que en test se aplique la distribución aprendida en train.

11.5 Agente de Momentum — `MomentumAgent`

Archivo: `module/agents/momentum_agent.py`

- **Modelo:** `RandomForestClassifier` con `StandardScaler` previo (calibrado)
- **Salida:** Score [0, 1] donde **1 = momentum alcista → Outperform**

Features que usa (FEATURE_COLS):

```

rsi_14, rsi_28
macd, macd_signal, macd_histogram
sma_20, sma_50, sma_200
bollinger_pct
price_vs_52w_high, price_vs_52w_low
momentum_1m, momentum_3m, momentum_6m, momentum_12m
volatility_20d, volatility_60d, atr_14
volume_ratio_20, volume_ratio_50
vix, yield_curve, sp500_momentum_3m, sp500_momentum_12m ← macro

```

Además, el método `_engineer_features()` crea features derivados binarios antes del entrenamiento:

- `rsi_overbought` (RSI > 70), `rsi_oversold` (RSI < 30)
- `above_sma200` (precio > SMA 200)
- `macd_bullish` (MACD > señal)
- `momentum_quality` (momentum de 1m y 3m ambos positivos)
- `vol_expansion` (volatility_20d > volatility_60d)
- `high_vix_regime` (VIX > 25)
- `inverted_yield_curve` (yield_curve < 0)
- `cross_sma_20_50` (SMA 20 > SMA 50)

11.6 Agente de Riesgo — `BearAgent`

Archivo: `module/agents/bear_agent.py`

El `BearAgent` es el único agente cuyo score está **invertido**: **1 = ALTO RIESGO → Underperform**. Esta inversión se aplica automáticamente en el entrenamiento pasando `1 - y_train` como labels.

- **Modelo:** Híbrido: 50% reglas deterministas + 50% Random Forest
- **Weights:** `BEAR_RULE_WEIGHT = 0.5`, `BEAR_ML_WEIGHT = 0.5` (ajustables en `environment.py`)

Features que usa:

```
total_debt_yoy_growth, debt_equity, debt_to_ebitda
fcf_margin, current_ratio
consecutive_losses, revenue_yoy_growth (negativo = riesgo), interest_coverage
insider_net_shares_90d, insider_sell_ratio
eps_surprise_pct, eps_revision
```

Las 10 reglas de riesgo (capa determinista, método `_compute_risk_flags()`):

#	Condición	Señal de riesgo
1	<code>total_debt_yoy_growth > 0.20</code>	Deuda creciendo >20% YoY
2	<code>debt_equity > 3.0</code>	Deuda/Equity por encima de 3x
3	<code>debt_to_ebitda > 6.0</code>	Deuda/EBITDA por encima de 6x
4	<code>fcf_margin < 0</code>	FCF negativo
5	<code>current_ratio < 1.0</code>	Ratio de liquidez < 1 (pasivo corriente > activo corriente)
6	<code>consecutive_losses >= 2</code>	2 o más trimestres consecutivos con pérdidas
7	<code>revenue_yoy_growth < 0</code>	Caída de ingresos YoY
8	<code>interest_coverage < 1.5</code>	El EBIT no llega a cubrir 1.5x los intereses
9	<code>insider_sell_ratio > 0.7</code>	Insiders vendiendo >70% de sus transacciones
10	<code>eps_surprise_pct < 0</code>	EPS por debajo de las estimaciones

Cada regla contribuye con **1/10** al score de riesgo de la capa de reglas. El score final es:

```
bear_score = BEAR_RULE_WEIGHT * rule_score + BEAR_ML_WEIGHT * ml_score
```

Hard threshold: Si `bear_score > BEAR_HARD_THRESHOLD (0.90)`, el meta-learner fuerza automáticamente un score de 0.2 (Underperform) para esa acción, independientemente de lo que digan los demás agentes.

11.7 Meta-Learner — **MetaLearner**

Archivo: `module/agents/meta_learner.py`

El meta-learner es el **árbitro final** que combina las señales de los 4 agentes. Implementa un esquema de stacking de dos niveles:

- **Nivel 0:** Los 4 agentes base generan sus scores
- **Nivel 1:** Un ensamble LR + GBM combina esos scores en una predicción final

Arquitectura del nivel 1:

El meta-learner entrena **dos modelos en paralelo** y elige el que mejor AUC obtenga en CV:

1. **Logistic Regression** (`C = META_LR_C = 0.5`): modelo lineal, interpretable
2. **GradientBoosting** (`n_estimators = 150, max_depth = 3`): captura interacciones no lineales

Features de entrada al meta-learner:

<code>fundamental_score</code>	← score del FundamentalAgent [0,1]
<code>valuation_score</code>	← score del ValuationAgent [0,1]
<code>momentum_score</code>	← score del MomentumAgent [0,1]
<code>bear_score</code>	← score del BearAgent [0,1] (INVERTIDO: 1=riesgo)
<code>vix</code>	← indicador de volatilidad macro
<code>yield_curve</code>	← spread tipos largo-corto
<code>sp500_momentum_3m</code>	← momentum 3m del índice
<code>sp500_momentum_12m</code>	← momentum 12m del índice
<code>sector_<nombre></code>	← dummies de sector (una columna por sector)

Capa de reglas del meta-learner:

Antes de devolver el score final, se aplica la regla hard del BearAgent:

```
if bear_score > BEAR_HARD_THRESHOLD: # 0.90
    final_score = 0.2 # Forzar Underperform
```

Salida: Score [0, 1] final donde **1 = Outperform**. Si `final_score >= 0.5`, la acción se clasifica como "Outperform" (`label = 1`).

11.8 OOF anti-leakage

Archivo: `module/pipeline/trainer.py` → función `_generate_oof_scores()`

El meta-learner necesita entrenarse con scores de los agentes base, pero si esos scores se calculan sobre los mismos datos de train, el meta-learner aprende correlaciones espurias (leakage).

La solución es **Out-Of-Fold (OOF)**: para cada agente base, se entrena en K-1 folds internos del train y se predice en el fold restante. Así, cada observación de train tiene un score calculado por un modelo que **nunca vio esa observación**.

```
Train set (N observaciones)
├─ KFold split 1: train en folds 2+3, predict en fold 1
├─ KFold split 2: train en folds 1+3, predict en fold 2
├─ KFold split 3: train en folds 1+2, predict en fold 3
    → OOF scores para todo el train (sin leakage)
```

Por qué KFold con `shuffle=True`: El dataset de train tiene observaciones de múltiples tickers en diferentes fechas (estructura panel, no serie temporal única). El orden temporal está garantizado por el walk-forward externo (todo el train precede al test). Dentro del train, mezclar mejora la representación sectorial en cada fold interno y evita que un fold KFold contenga solo empresas de un sector específico o un período concreto.

11.9 Backtester y métricas

Archivo: `module/backtester.py` → clase `WalkForwardBacktester`

Después de obtener los scores del test, el backtester simula la cartera:

- 1. **Selección de cartera:** Top `TOP_N_STOCKS = 10` acciones por score `final_score`
- 2. **Pesos:** Equal weight (1/10 por acción) — se puede cambiar
- 3. **Retornos diarios:** Para cada día del período de test, promedio de retornos de las acciones en cartera
- 4. **Comparación vs benchmark:** El benchmark es el S&P 500 (`^GSPC`)

Métricas calculadas por fold:

Métrica	Descripción
<code>cumulative_return</code>	Retorno total del período (%)
<code>annualized_return</code>	Retorno anualizado (%)
<code>sharpe_ratio</code>	Retorno ajustado por riesgo (usando <code>RISK_FREE_RATE = 0.04</code>)
<code>sortino_ratio</code>	Como Sharpe pero solo penaliza volatilidad negativa
<code>max_drawdown</code>	Máxima caída desde máximo acumulado (%)
<code>calmar_ratio</code>	Retorno anualizado / max_drawdown
<code>hit_rate</code>	% de señales "Outperform" que realmente batieron al benchmark
<code>alpha</code>	Retorno de estrategia - retorno benchmark en el mismo período
<code>auc_roc</code>	AUC del clasificador (calidad de ranking, no de cartera)

Todos los resultados se guardan en `results/backtest/fold_<N>_metrics.json`.

11.10 Explicabilidad SHAP

Archivo: `module/explainer.py` → clase `AgentExplainer` **Invocado desde:**

`module/pipeline/evaluator.py` → función `explain_top_tickers()`

Para cada fold, se generan explicaciones SHAP de los **10 mejores y 10 peores** tickers (por `final_score`).

Para cada ticker seleccionado:

1. Se toma la última observación del período de test
2. Para cada agente (fundamental, valuation, momentum, bear):
 - Si el agente tiene un `_explainer` asociado → `ag._explainer.explain_prediction(row, ticker, score)`
 - El explainer usa SHAP `TreeExplainer` para calcular los valores SHAP
 - Devuelve los 6 features más influyentes con su contribución y dirección
3. Se genera un texto explicativo human-readable

Salida: `results/agents/fold_<N>_ticker_explanations.json`

```
{
  "fold": 1,
  "tickers": {
    "AAPL": {
      "score": 0.823,
      "label": "Outperform",
      "agents": {
        "fundamental": {
          "text": "Alta rentabilidad (ROE=0.45) impulsa el score...",
          "top_drivers": [
            {"feature": "roe_zsector", "shap_value": 0.12, "feature_value": 1.4,
              "value_formatted": "1.40", "rank": 1},
            ...
          ]
        }
      }
    }
  }
}
```

La función `save_global_explanation()` genera el JSON de importancias por agente. Además del JSON de importancias, también genera `shap_bar<suffix>.png`: un gráfico de barras horizontal con las 15 features más importantes según SHAP.

11.11 Ablation study

Archivo: `module/pipeline/ablation.py` → funciones `run_ablation_study()` y `summarize_ablation()`

Invocado desde: `module/pipeline/evaluator.py` (dentro del loop de folds)

Evalúa la **contribución marginal de cada agente** al meta-learner: ¿cuánto baja el AUC si eliminamos un agente?

Metodología por fold:

1. Tomar las columnas `*_score` OOF del train (generadas sin leakage en `train_fold`)
2. Entrenar una Logistic Regression sencilla con todos los scores → AUC baseline
3. Repetir excluyendo cada agente por turno → AUC sin ese agente
4. `marginal_contribution = baseline_auc - auc_without`

Archivos generados:

- `results/agents/ablation_fold<N>.json` — resultado por fold
- `results/agents/ablation_summary.json` — media, std y % de folds positivos por agente
- `results/agents/ablation_summary.csv` — tabla legible

Nota: el ablation usa una LR sencilla (no el GBM completo del MetaLearner) para mantener el análisis rápido y reproducible.

12. Paso 8 — Fold live out-of-sample

Archivo: `module/pipeline/live_fold.py` → función `run_live_fold()`

Este paso evalúa el sistema en datos **completamente fuera de muestra**: predicciones sobre Q1 2026 usando modelos entrenados sobre 2021-2025.

Flujo del fold live

1. `build_live_features(tickers, as_of_date="2026-01-01", router, builders)`
 - Construye features para cada ticker a fecha 2026-01-01
 - Sin label (no hay retorno futuro conocido en el momento de predecir)
2. `train_full_history(df_norm, y, agents_results_dir, random_seed)`
 - Entrena todos los agentes en TODO el histórico 2021-2025
 - No se puede hacer OOF aquí (no hay "pasado del pasado" para el meta-learner)
 - El meta-learner se entrena directamente con los scores del train completo
3. Predicción de scores para todos los tickers vivos
 - `final_score` por ticker
 - `label = "Outperform"` si `final_score >= 0.5`
4. `_download_live_prices(tickers, start="2026-01-01", end=hoy)`
 - Descarga precios reales de Q1 2026 en memoria (no se guardan en disco)
 - Usa `yfinance` directamente
5. `_qtd_return(close_series)` para cada ticker
 - Retorno desde primer día disponible hasta el último
6. Comparar predicciones vs retornos reales
 - `hit_rate`: % de "Outperform" que realmente superaron la mediana
 - `alpha`: retorno medio de los predichos Outperform - mediana del universo

```
7. Guardar resultados:
  → results/live_predictions.csv
  → results/live_evaluation.json
```

Diferencia con el backtest histórico

Característica	Walk-Forward Backtest	Live Fold
Datos de test	Histórico pasado	Datos de 2026 (reales)
Labels	Calculados de histórico	Calculados de precios descargados en el momento
OOF	Sí (anti-leakage)	No (se usa todo el histórico para entrenar)
Propósito	Evaluar el método históricamente	Validar en datos nunca vistos

13. Resultados y ficheros de salida

Todos los resultados se generan en `results/`:

Archivo	Descripción
<code>results/master_dataset.csv</code>	Dataset completo (ticker, date, features, forward_return)
<code>results/pipeline.log</code>	Log completo de la ejecución con timestamps
<code>results/agents/<agente>/diagnostics_fold<N>.json</code>	Métricas de entrenamiento por fold (AUC, n_samples, balance de clases, ...)
<code>results/agents/<agente>/feature_importances_fold<N>.csv</code>	Importancia de features ordenadas
<code>results/agents/<agente>/predictions_fold<N>.csv</code>	Predicciones del agente en el test del fold
<code>results/agents/<agente>/train_history.json</code>	Historial de métricas de todos los folds entrenados
<code>results/agents/fold_<N>_ticker_explanations.json</code>	Explicaciones SHAP por ticker
<code>results/agents/all_folds_diagnostics.json</code>	Diagnósticos consolidados de todos los folds
<code>results/agents/ablation_fold<N>.json</code>	Contribución marginal de cada agente en el fold N
<code>results/agents/ablation_summary.json</code>	Resumen del ablation study (media ± std por agente)

Archivo	Descripción
results/agents/ablation_summary.csv	Tabla del ablation estudio (una fila por agente)
results/agents/shap_bar<agente>_fold<N>.png	Gráfico de barras SHAP (top 15 features) por agente y fold
results/backtest/fold_<N>_metrics.json	Métricas de cartera del fold (alpha, Sharpe, drawdown, ...)
results/backtest/folds_summary.csv	Resumen de todos los folds en una tabla
results/backtest/report.txt	Informe textual completo del backtest (métricas globales + tabla de folds)
results/plots/full_report.png	Gráfico completo (7 subplots) de rendimiento
results/plots/score_distribution_fold<N>.png	Distribución de scores por fold
results/plots/feature_importances_<agente>_fold<N>.png	Gráfico de feature importances
results/live_predictions.csv	Predicciones del fold live por ticker
results/live_evaluation.json	Evaluación del fold live (hit_rate, alpha, ...)

Gráficos del informe completo (`full_report.png`, 7 subplots):

- 1. Curvas de riqueza: estrategia vs S&P 500
- 2. Drawdown de la estrategia
- 3. Alpha acumulado
- 4. Retorno anualizado por fold
- 5. Distribución de retornos mensuales
- 6. Sharpe por fold
- 7. AUC por agente (comparación de los 5 modelos)

14. Mecanismos anti-leakage

Este sistema implementa múltiples capas de protección contra el data snooping:

1. Filing Lag (+45 días)

Los fundamentales de Q1 (cerrado 31-Mar) no son visibles al modelo hasta el 15-May aproximadamente. `DataRouter.load_consolidated()` desplaza el índice temporal +45 días. Si se cambia `FILING_LAG_DAYS`, se modifica cuándo el modelo "ve" los datos.

2. Walk-Forward (separación temporal estricta)

El test de cada fold es siempre posterior a su train. No hay datos de test que contaminen el train. La ventana de test (1 trimestre) nunca se solapa con la ventana de train.

3. OOF para el meta-learner

Los scores que el meta-learner usa para entrenarse se generan con modelos que **nunca vieron** esa observación. Ver sección 11.8 para detalle.

4. Normalización sectorial fit-only-on-train

El `SectorNormalizer` aprende medias y desviaciones estándar solo de los datos de entrenamiento. En test, aplica la transformación ya fija. Esto evita que la distribución del test influya en la normalización.

5. Percentiles sectoriales de valoración

Los percentiles `pe_sector_pct`, etc., se calculan en `ValuationAgent.fit()` usando solo empresas del train. En `predict_score()` se aplica el ranking aprendido en train.

6. Labels con mediana del train

La mediana que binariza el label (`forward_return > median_ret`) se calcula **siempre** sobre el train del fold. La misma mediana se aplica al test, pero no se recalcula con datos del test.

7. Precios live solo para evaluación

En el fold live, los precios de Q1 2026 se descargan únicamente para calcular los retornos reales después de las predicciones. No influyen en el entrenamiento.

15. Cómo modificar el sistema

Cambiar el universo de acciones

Editar la función `fetch_tickers()` en `module/fetcher.py` para añadir o quitar tickers. Después ejecutar con `FORCE_DOWNLOAD = True` para descargar los nuevos datos.

Cambiar el período de análisis

Modificar `START_DATE` y/o `END_DATE` en `environment.py`.

Cambiar hiperparámetros de un agente

Modificar las constantes correspondientes en `environment.py` (sección 7). No tocar el código del agente.

Cambiar el horizonte de predicción

Modificar `FORWARD_RETURN_DAYS` en `environment.py`. Asegurarse de que `WALKFORWARD_TEST_QUARTERS` sea coherente (1 quarter $\approx$ 63 días de trading).

Añadir un nuevo agente

1. Crear `module/agents/nuevo_agente.py` heredando de `BaseAgent`

2. Implementar `fit(X, y, fold, **kwargs)` y `predict_score(X)`
3. Añadir el agente a `train_fold()` en `module/pipeline/trainer.py`
4. Añadir sus scores como features del meta-learner en `MetaLearner`
5. Añadir sus hiperparámetros a `environment.py`

Añadir nuevas features

1. Añadir el cálculo al builder correspondiente en `module/feature_engineering.py`
2. Añadir el nombre de la columna a `FEATURE_COLS` del agente que la va a usar
3. Si la feature necesita normalización sectorial, añadirla al bloque de `SectorNormalizer`

Cambiar el tamaño de la cartera

Modificar `TOP_N_STOCKS` en `environment.py`.

Deshabilitar el backtest (ejecución rápida)

Poner `SKIP_BACKTEST = True` en `environment.py`. Ejecutará solo el fold live.

Forzar re-descarga de datos

Poner `FORCE_DOWNLOAD = True` en `environment.py`.

Regenerar los fundamentales consolidados

Descomentar la línea `# prepare_data(tickers, data_dir=DATA_DIR)` en `analyzer.py` (Paso 2) y ejecutar. Volver a comentarla después para no ralentizar ejecuciones futuras.

Ejecutar con un subconjunto de tickers (prueba rápida)

Usar `run_pipeline.py` con el flag `--tickers`:

```
python run_pipeline.py --tickers AAPL MSFT NVDA GOOGL --skip-download
```

Instalar dependencias

```
pip install -r requirements.txt
```

Interpretar el ablation study

Los ficheros `results/agents/ablation_summary.csv` y `.json` muestran la contribución marginal de cada agente. Un `mean_contribution > 0` significa que el agente añade valor; `< 0` indica que ese agente degrada el ensemble en promedio. `pct_folds_positive` indica en qué fracción de folds el agente contribuyó positivamente.

Última actualización: 2026-03-18. Para cualquier modificación, actualizar este documento junto con el código.